BDI API Contracts Documentation

Overview

This document defines the API contracts for the Binary Decomposition Interface (BDI) C implementation. These contracts establish the expected behavior, preconditions, postconditions, and invariants for all public APIs.

Core Principles

Memory Management

- **Ownership**: All functions that allocate memory must clearly document ownership transfer
- **Null Safety**: Functions must specify whether they accept nullptr parameters
- **Cleanup**: All allocated resources must have a corresponding cleanup function

Error Handling

- **Return Codes**: Functions returning error codes must use consistent error code conventions
- **nullptr Returns**: Functions returning pointers must document nullptr return conditions
- **Error Propagation**: Errors must be propagated up the call stack consistently

Thread Safety

- **Concurrency**: Document whether functions are thread-safe
- **Synchronization**: Specify required synchronization primitives
- **Reentrancy**: Indicate if functions are reentrant

Module API Contracts

BCI (Binary Computational Interface)

`chimera_bci.h`

Function: `bci_init()`
- **Purpose**: Initialize the BCI subsystem
- **Preconditions**: Must be called before any other BCI functions
- **Postconditions**: BCI subsystem is ready for use
- **Return**: 0 on success, negative error code on failure
- **Thread Safety**: Not thread-safe, must be called from main thread only
- **Error Codes**:
- `-1`: Initialization failed
- `-2`: Already initialized

Function: `bci_cleanup()`
- **Purpose**: Clean up BCI subsystem resources
- **Preconditions**: `bci_init()` must have been called successfully
- **Postconditions**: All BCI resources are freed
- **Return**: void
- **Thread Safety**: Not thread-safe, must be called after all BCI operations complete

BTL (Binary Translation Layer)

`chimera_btl.h`

Function: `btl_translate()`
- **Purpose**: Translate binary code to intermediate representation
- **Preconditions**:
- Input buffer must not be nullptr
- Input size must be > 0
- **Postconditions**: Returns translated code or nullptr on error
- **Return**: Pointer to translated code, or nullptr on failure
- **Memory**: Caller must free returned pointer using `btl_free_translation()`
- **Thread Safety**: Thread-safe

Compiler Module

`bci_lexer.h`

Function: `lexer_create(const char* source)`
- **Purpose**: Create a new lexer instance
- **Preconditions**: source must not be nullptr
- **Postconditions**: Returns initialized lexer or nullptr
- **Return**: Lexer pointer or nullptr on allocation failure
- **Memory**: Caller must call `lexer_destroy()` to free
- **Thread Safety**: Thread-safe

Function: `lexer_next_token(Lexer* lexer)`
- **Purpose**: Get next token from input stream
- **Preconditions**: lexer must not be nullptr
- **Postconditions**: Returns next token or EOF token
- **Return**: Token structure
- **Thread Safety**: Not thread-safe per lexer instance

`bci_parser.h`

Function: `parser_create(Lexer* lexer)`
- **Purpose**: Create a new parser instance
- **Preconditions**: lexer must not be nullptr and initialized
- **Postconditions**: Returns initialized parser or nullptr
- **Return**: Parser pointer or nullptr on allocation failure
- **Memory**: Caller must call `parser_destroy()` to free
- **Thread Safety**: Thread-safe

Function: `parser_parse(Parser* parser)`
- **Purpose**: Parse input and generate AST
- **Preconditions**: parser must not be nullptr
- **Postconditions**: Returns AST root or nullptr on parse error
- **Return**: AST node pointer or nullptr
- **Memory**: Caller must call `ast_destroy()` to free AST
- **Thread Safety**: Not thread-safe per parser instance

`bci_analyzer.h`

Function: `analyzer_create()`
- **Purpose**: Create semantic analyzer instance
- **Preconditions**: None

- **Postconditions**: Returns initialized analyzer or nullptr
- **Return**: Analyzer pointer or nullptr on allocation failure
- **Memory**: Caller must call `analyzer_destroy()` to free
- **Thread Safety**: Thread-safe

Function: `analyzer_analyze(Analyzer* analyzer, ASTNode* ast)`
- **Purpose**: Perform semantic analysis on AST
- **Preconditions**:
- analyzer must not be nullptr
- ast must not be nullptr
- **Postconditions**: Returns 0 on success, error code on failure
- **Return**: 0 on success, negative error code on failure
- **Thread Safety**: Not thread-safe per analyzer instance

Kernel Module

`device.h`

Function: `device_init()`
- **Purpose**: Initialize device subsystem
- **Preconditions**: Must be called before device operations
- **Postconditions**: Device subsystem ready
- **Return**: 0 on success, negative on failure
- **Thread Safety**: Not thread-safe

Function: `device_execute(DeviceType type, void** inputs, void** outputs)`
- **Purpose**: Execute operation on specified device
- **Preconditions**:
- Device subsystem must be initialized
- inputs and outputs must not be nullptr
- **Postconditions**: Operation executed on device
- **Return**: 0 on success, negative on failure
- **Thread Safety**: Thread-safe

`scheduler.h`

Function: `scheduler_init()`
- **Purpose**: Initialize scheduler subsystem
- **Preconditions**: None
- **Postconditions**: Scheduler ready to accept tasks
- **Return**: 0 on success, negative on failure
- **Thread Safety**: Not thread-safe

Function: `scheduler_schedule_task(Task* task)`
- **Purpose**: Schedule task for execution
- **Preconditions**:
- Scheduler must be initialized
- task must not be nullptr
- **Postconditions**: Task queued for execution
- **Return**: Task ID on success, negative on failure
- **Thread Safety**: Thread-safe

VM Module

`bci_vm.h`

Function: `vm_create()`
- **Purpose**: Create new VM instance
- **Preconditions**: None
- **Postconditions**: Returns initialized VM or nullptr
- **Return**: VM pointer or nullptr on allocation failure
- **Memory**: Caller must call `vm_destroy()` to free
- **Thread Safety**: Thread-safe

Function: `vm_execute(VM* vm, Chunk* chunk)`
- **Purpose**: Execute bytecode chunk
- **Preconditions**:
- vm must not be nullptr
- chunk must not be nullptr
- **Postconditions**: Bytecode executed
- **Return**: 0 on success, negative on failure
- **Thread Safety**: Not thread-safe per VM instance

Error Code Conventions

Standard Error Codes

```
#define BDI_SUCCESS           0
#define BDI_ERROR_GENERIC    -1
#define BDI_ERROR_NOMEM      -2
#define BDI_ERROR_INVALID    -3
#define BDI_ERROR_NOTFOUND   -4
#define BDI_ERROR_TIMEOUT    -5
#define BDI_ERROR_IO         -6
#define BDI_ERROR_PERMISSION -7
```

Module-Specific Error Codes

Each module may define additional error codes starting from -100:
- BCI: -100 to -199
- BTL: -200 to -299
- Compiler: -300 to -399
- Kernel: -400 to -499
- VM: -500 to -599

Invariants

Global Invariants

1. All public functions validate their parameters
2. All allocated memory is tracked and can be freed
3. All error conditions are reported through return values
4. No function modifies global state without synchronization

Module Invariants

Compiler

1. Lexer always produces valid tokens or EOF
2. Parser always produces valid AST or reports error
3. Semantic analyzer never modifies AST structure

Kernel

1. Device operations are atomic
2. Scheduler maintains task ordering guarantees
3. Memory allocations are aligned to device requirements

VM

1. Stack never overflows
2. All bytecode instructions are valid
3. VM state is consistent after each instruction

Deprecation Policy

When APIs need to change:
1. Mark old API as deprecated with `[[deprecated]]` attribute
2. Provide migration path in documentation
3. Maintain old API for at least 2 major versions
4. Log warnings when deprecated APIs are used

Version Compatibility

- **Major Version**: Breaking API changes
- **Minor Version**: New features, backward compatible
- **Patch Version**: Bug fixes only

Current Version: 1.0.0 (C23 Modernization)